

UNED IA 2026/27

# ESTRUCTURAS DE DATOS Y ALGORITMOS

Código 71032015

## ELEGIR CÓMO GUARDAR ES ELEGIR CUÁNTO CUESTA

Aprenderás a transformar datos en arrays, listas, tablas hash, pilas, colas, árboles, montículos y grafos, y a justificar qué operaciones son rápidas o lentas.

**Manual completo desde cero · 5 temas · pseudocódigo · ejercicios · desarrollo**

# Antes de empezar

## LA IDEA CENTRAL

Una estructura de datos decide cómo se organizan los datos; un algoritmo decide los pasos que hacemos con ellos. La pareja correcta permite resolver el problema con claridad, tiempo y memoria razonables.

## Mapa oficial convertido en camino comprensible

| Tema            | Contenido oficial                              | Pregunta humana                                        |
|-----------------|------------------------------------------------|--------------------------------------------------------|
| 1. Recursión    | Noción, algoritmos y estructuras recursivas    | ¿Cómo resuelvo una versión pequeña del mismo problema? |
| 2. Coste        | Modelo RAM, función temporal, Landau y cálculo | ¿Cómo crecerá el trabajo al crecer la entrada?         |
| 3. Indexadas    | Arrays, tablas hash y conjuntos                | ¿Cómo encuentro directamente por posición o clave?     |
| 4. Secuenciales | Listas, pilas, colas y deque                   | ¿En qué orden entran, salen y se recorren?             |
| 5. Jerárquicas  | Árboles, BST, montículos y grafos              | ¿Cómo represento niveles y conexiones?                 |

## Formato oficial 2026/27

| Elemento           | Dato verificado                                                                                                                            |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Asignatura         | 2.º curso · 1.º semestre · 6 ECTS · 150 horas                                                                                              |
| Recomendadas antes | Fundamentos de Programación, Lógica y Estructuras Discretas, y Programación Orientada a Objetos                                            |
| Examen             | Desarrollo · 120 minutos · sin material. La guía pública no fija número de preguntas                                                       |
| Criterios          | Corrección, completitud, relevancia y justificación                                                                                        |
| Escaneo            | Bolígrafo, nunca lápiz; dejar margen para evitar pérdida de texto                                                                          |
| Práctica           | Obligatoria, 20%; debe funcionar y superar juegos de prueba                                                                                |
| Nota               | Práctica suspensa/no entregada: final 0. Con práctica aprobada y examen <4: final=0,8·examen. Con examen ≥4: final=0,8·examen+0,2·práctica |

## LÍMITE ACADÉMICO IMPORTANTE

La guía prohíbe usar IA generativa para elaborar trabajos académicos de la asignatura salvo autorización expresa del equipo docente. Este manual es para estudiar y practicar; no contiene ni sustituye la práctica obligatoria.

# Puente previo: lo mínimo de programación

Esta asignatura no empieza completamente desde cero. Antes de estudiar estructuras complejas conviene reconocer estas piezas. Si alguna resulta nueva, no la memorices: ejecuta los ejemplos a mano.

| Idea         | Significado sencillo                                   | Ejemplo                              |
|--------------|--------------------------------------------------------|--------------------------------------|
| Variable     | Nombre asociado a un valor                             | $n=5$                                |
| Función      | Bloque que recibe datos y devuelve un resultado        | máximo(a,b)                          |
| Condición    | Elige qué pasos ejecutar                               | si $n=0$                             |
| Bucle        | Repite pasos mientras se cumpla algo                   | para $i=0..n-1$                      |
| Referencia   | Forma de llegar a un objeto o nodo                     | siguiente                            |
| Clase/objeto | Molde y ejemplar con datos y operaciones               | Nodo(dato,enlace)                    |
| Invariante   | Propiedad que sigue siendo cierta durante el algoritmo | La parte izquierda ya está procesada |

## Pseudocódigo que usaremos

```
función buscar(a, objetivo):  
    para i desde 0 hasta longitud(a)-1:  
        si a[i] = objetivo:  
            devolver i  
    devolver NO_ENCONTRADO
```

No es un lenguaje concreto. La sangría muestra qué pasos pertenecen al bucle o a la condición. La práctica usará las herramientas que indique el curso virtual.

### TAD: TIPO ABSTRACTO DE DATOS

Un TAD define **qué operaciones ofrece**, no cómo están hechas. Una pila ofrece apilar, desapilar y cima; puede implementarse con array o lista. Separar interfaz e implementación permite cambiar por dentro sin romper a quien la usa.

# Base para leer cualquier algoritmo

## Las cinco preguntas

- ¿Qué representa cada dato?
- ¿Qué condición hace entrar o salir?
- ¿Qué cambia en cada paso?
- ¿Por qué termina?
- ¿Qué propiedad demuestra que el resultado es correcto?

### EJEMPLO: BÚSQUEDA LINEAL

Entrada:  $a=[8,3,9,5]$  y objetivo=9. El índice  $i$  señala la posición actual.

$i=0$ : 8 no es 9.  $i=1$ : 3 no es 9.  $i=2$ : 9 coincide; devolvemos 2 y terminamos.

Si recorremos todo sin coincidencia, devolvemos NO\_ENCONTRADO.

Invariante: antes de examinar  $i$ , el objetivo no está en las posiciones anteriores.

## Traza manual

| Paso | $i$ | $a[i]$ | Decisión   |
|------|-----|--------|------------|
| 1    | 0   | 8      | continuar  |
| 2    | 1   | 3      | continuar  |
| 3    | 2   | 9      | devolver 2 |

## Ahora tú

### HAZLO SIN MIRAR

1. ¿Qué devuelve si busca 5?
2. ¿Cuántas comparaciones hace al buscar 4?
3. ¿Qué garantiza la invariante antes de  $i=2$ ?

### CORRECCIÓN EXPLICADA

1. Devuelve 3.
2. Cuatro; 4 no aparece y revisa todo.
3. Que 9 no está en las posiciones 0 y 1 ya descartadas.

## TEMA 1

# Recursión

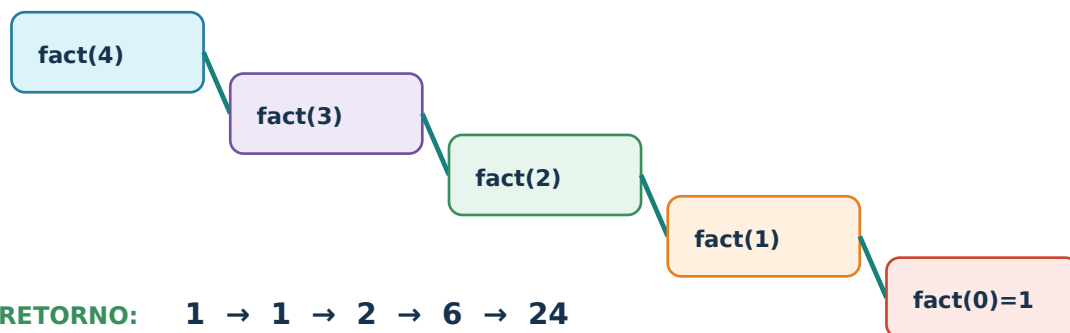
## LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo puede una función resolver un problema llamándose con una entrada más pequeña?

## AL TERMINAR PODRÁS

Trazar llamadas y retornos, reconocer caso base y progreso, y justificar terminación y corrección.

**factorial(4): primero baja, después devuelve resultados**



Cada llamada conserva n y la multiplicación pendiente en la pila.

## 1.1 Las tres condiciones

| Pieza          | Pregunta                                         |
|----------------|--------------------------------------------------|
| Caso base      | ¿Qué entrada sencilla resuelvo sin otra llamada? |
| Paso recursivo | ¿Cómo uso la solución de un problema menor?      |
| Progreso       | ¿Por qué cada llamada se acerca al caso base?    |

```

función factorial(n):
  si n = 0: devolver 1
  devolver n * factorial(n-1)
  
```

### NO OCURRE TODO A LA VEZ

factorial(4) no conoce 24 al llamar a factorial(3). Deja pendiente multiplicar por 4. Cada marco guarda su n y el trabajo pendiente hasta que la llamada inferior devuelve.

## 1.2 Recursión sobre un array

```
función sumar(a, n):  
    si n = 0: devolver 0  
    devolver a[n-1] + sumar(a, n-1)  
  
sumar([3,5,2],3) = 2 + sumar(a,2)  
                 = 2 + 5 + sumar(a,1)  
                 = 2 + 5 + 3 + sumar(a,0) = 10
```

La entrada  $n$  disminuye hasta 0. Hay  $n+1$  llamadas contando el caso base. Tiempo  $O(n)$  y espacio de pila  $O(n)$ .

### 1.3 Búsqueda binaria recursiva

Solo funciona en una colección ordenada. Compara con el centro y descarta la mitad donde el objetivo no puede estar.

```
binaria(a, x, izq, der):  
    si izq > der: devolver NO_ENCONTRADO  
    m = (izq + der) // 2  
    si a[m] = x: devolver m  
    si x < a[m]: devolver binaria(a,x,izq,m-1)  
    devolver binaria(a,x,m+1,der)
```

#### BUSCAR 23 EN [4,8,15,16,23,42]

Rango 0..5, centro 2, valor 15. Como  $23 > 15$ , queda 3..5.

Centro 4, valor 23. Encontrado en el índice 4.

Cada comparación divide el rango aproximadamente entre dos:  $O(\log n)$ .

### 1.4 Demostrar que es correcta

Una demostración recursiva suele seguir la misma forma: base correcta; suponemos correcta la llamada menor; explicamos por qué el paso actual produce la respuesta correcta. Además demostramos progreso para excluir llamadas infinitas.

### 1.5 Estructuras recursivas

Una lista enlazada puede definirse como vacía o como un nodo seguido de otra lista. Un árbol es una raíz con subárboles. La estructura y el algoritmo recursivo encajan: procesar nodo actual y llamar sobre las partes menores.

## Ahora tú

### HAZLO SIN MIRAR

1. Caso base de longitud de una lista enlazada.
2. ¿Por qué factorial(-1) sería peligroso con el código anterior?
3. Coste temporal y espacial de sumar  $n$  elementos recursivamente.
4. ¿Qué requisito necesita búsqueda binaria?
5. ¿Cuántos marcos crea factorial(4) contando el base?

### CORRECCIÓN EXPLICADA

1. Lista vacía devuelve 0.
2.  $n$  se aleja de 0; no se cumple el progreso y no termina.
3. Tiempo  $O(n)$ , pila  $O(n)$ .
4. La colección debe estar ordenada y los límites deben reducirse.
5. Cinco:  $n=4,3,2,1,0$ .

## TEMA 2

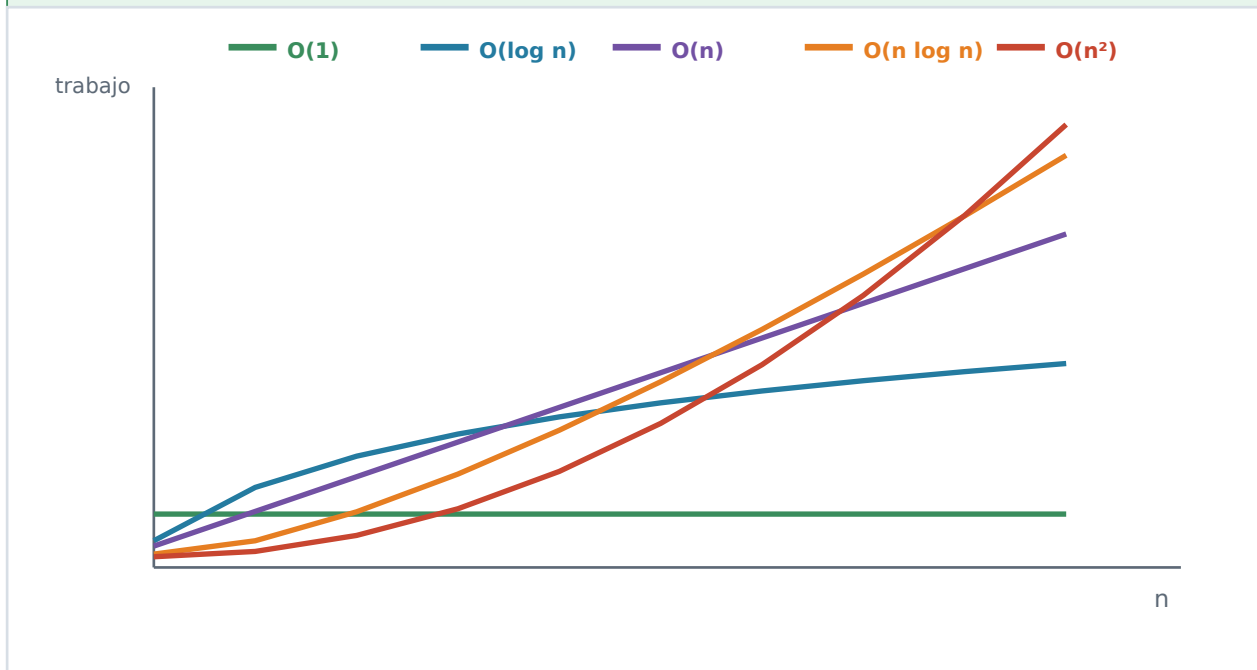
# Introducción al coste computacional

## LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cómo sabemos si un algoritmo seguirá siendo útil cuando  $n$  sea enorme?

## AL TERMINAR PODRÁS

Construir  $T(n)$ , usar  $O$ ,  $\Omega$  y  $\Theta$ , analizar bucles y recursiones, y distinguir tiempo de espacio.



## 2.1 Modelo RAM y tamaño de entrada

El modelo RAM simplifica el computador: operaciones elementales como asignar, comparar o sumar cuestan una unidad. No predice segundos exactos; permite comparar cómo crece el número de pasos.

| Problema            | $n$ suele significar                      |
|---------------------|-------------------------------------------|
| Buscar en array     | número de elementos                       |
| Procesar texto      | número de caracteres                      |
| Matriz $n \times n$ | $n$ puede ser dimensión; hay $n^2$ celdas |
| Grafo               | vértices $V$ y aristas $E$                |

## 2.2 Mejor, peor y promedio

Buscar linealmente puede acertar en la primera posición - mejor  $\Theta(1)$  - o revisar  $n$  posiciones - peor  $\Theta(n)$ . El promedio necesita un modelo de probabilidad; no se debe inventar sin indicarlo.

## 2.3 $O$ , $\Omega$ y $\Theta$

| Notación  | Qué limita | Lectura útil                                         |
|-----------|------------|------------------------------------------------------|
| $O(g(n))$ | Por arriba | No crece más deprisa que ese orden, salvo constantes |

| Notación       | Qué limita      | Lectura útil                  |
|----------------|-----------------|-------------------------------|
| $\Omega(g(n))$ | Por abajo       | Necesita al menos ese orden   |
| $\Theta(g(n))$ | Por ambos lados | Crece realmente con ese orden |

**SIMPLIFICAR SIN PERDER LO IMPORTANTE**

$3n^2+7n+20$  es  $\Theta(n^2)$ . Se conserva el término que crece más y se ignoran constantes multiplicativas. Esto describe crecimiento, no afirma que  $3n^2$  y  $n^2$  hagan exactamente los mismos pasos.

# Calcular costes paso a paso

## Secuencia, alternativa y repetición

| Estructura                     | Regla                                            |
|--------------------------------|--------------------------------------------------|
| Bloques consecutivos           | Suma costes; domina el mayor orden               |
| if/else                        | Coste condición + máximo de ramas para peor caso |
| Bucle n veces                  | $n \cdot \text{coste del cuerpo}$                |
| Bucles anidados independientes | Multiplica repeticiones                          |
| Entrada se divide por 2        | Número de iteraciones $\log_2 n$                 |

```

para i=0..n-1:           -> n veces ->  $\theta(n)$ 
    usar(a[i])

para i=0..n-1:           ->  $n \cdot n$  ->  $\theta(n^2)$ 
    para j=0..n-1:
        comparar(a[i],a[j])

i=1                       -> 1,2,4,8,... ->  $\theta(\log n)$ 
mientras i<n:
    i=2*i
  
```

## Bucle triangular

Si el bucle interior se repite  $i$  veces, el total es  $0+1+\dots+(n-1)=n(n-1)/2$ . Sigue siendo  $\theta(n^2)$ , aunque no ejecute exactamente  $n^2$  operaciones.

## Recurrencias sencillas

| Recurrencia      | Historia                         | Orden              |
|------------------|----------------------------------|--------------------|
| $T(n)=T(n-1)+c$  | Reduce uno cada vez              | $\theta(n)$        |
| $T(n)=T(n/2)+c$  | Reduce a la mitad                | $\theta(\log n)$   |
| $T(n)=2T(n/2)+n$ | Dos mitades y combinación lineal | $\theta(n \log n)$ |

## Espacio

Cuenta memoria adicional: variables constantes  $\theta(1)$ , array auxiliar de  $n$  elementos  $\theta(n)$ , matriz  $n \times n$   $\theta(n^2)$ , pila recursiva de profundidad  $n$   $\theta(n)$ . La entrada original normalmente se separa del espacio auxiliar.

## Ahora tú

### HAZLO SIN MIRAR

1. Coste de dos bucles consecutivos  $\Theta(n)$  y  $\Theta(n^2)$ .
2. Coste de tres bucles anidados de  $n$  iteraciones.
3. ¿Por qué dividir  $n$  por 2 produce  $\log n$ ?
4.  $T(n)=T(n-1)+1$ .
5. Espacio de una función recursiva con profundidad  $\log n$  y datos constantes por llamada.

### CORRECCIÓN EXPLICADA

1.  $\Theta(n+n^2)=\Theta(n^2)$ .
2.  $\Theta(n^3)$ .
3. Porque después de  $k$  divisiones  $n/2^k \approx 1$ ;  $k \approx \log_2 n$ .
4.  $\Theta(n)$ .
5.  $\Theta(\log n)$ .

## TEMA 3

# Estructuras de datos indexadas

## LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Cuándo conviene acceder por posición y cuándo por una clave?

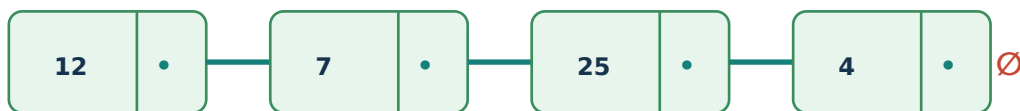
## AL TERMINAR PODRÁS

Comparar arrays, tablas hash y conjuntos, calcular índices y colisiones, y justificar sus costes.

### Array: casillas contiguas e índice directo



### Lista enlazada: cada nodo guarda dato y enlace



## 3.1 Arrays

Un array guarda elementos del mismo tipo lógico en posiciones consecutivas. Si conocemos el índice  $i$ , la dirección se calcula directamente:  $\text{base} + i \cdot \text{tamaño}$ . Por eso leer o escribir  $a[i]$  es  $\Theta(1)$ .

| Operación array           | Coste típico | Motivo               |
|---------------------------|--------------|----------------------|
| Acceso $a[i]$             | $\Theta(1)$  | Cálculo directo      |
| Buscar valor sin ordenar  | $\Theta(n)$  | Puede estar al final |
| Insertar/borrar en medio  | $\Theta(n)$  | Hay que desplazar    |
| Añadir al final con hueco | $\Theta(1)$  | No desplaza          |

### INSERTAR 9 EN ÍNDICE 2

Partimos de  $[4, 7, 12, 20]$  con capacidad suficiente.

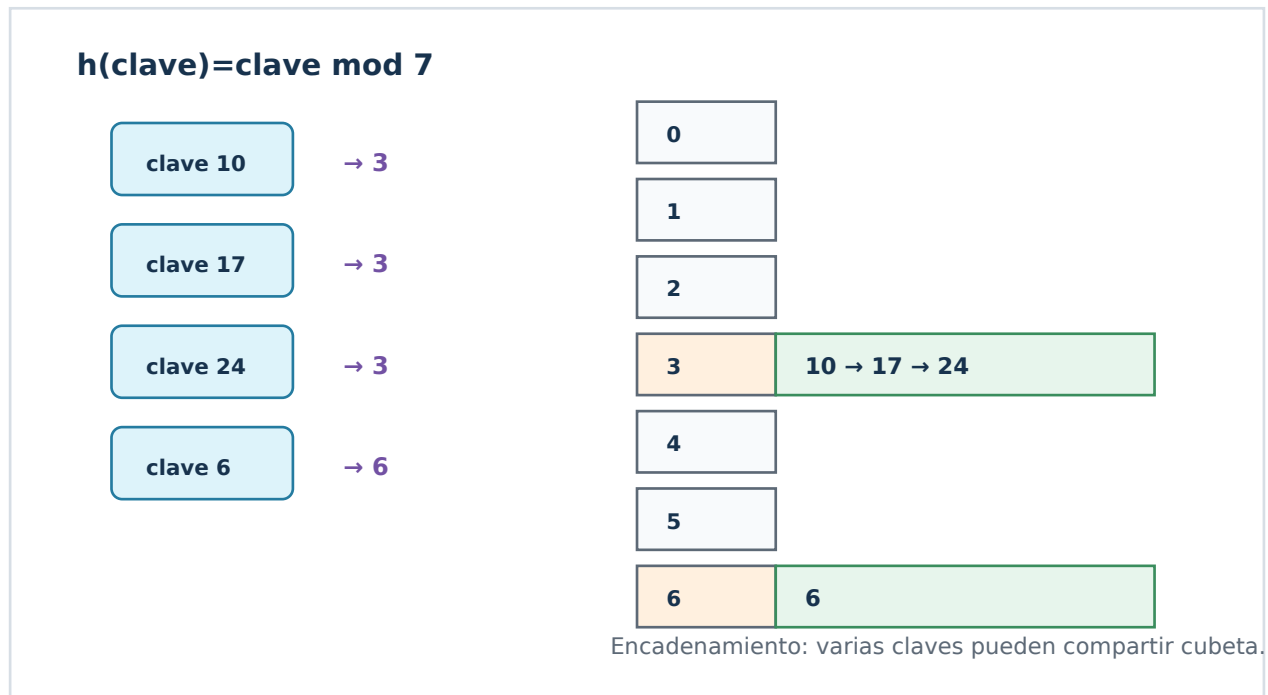
Desplazamos desde el final: 20 a 4, 12 a 3. Escribimos 9 en 2.

Resultado  $[4, 7, 9, 12, 20]$ . Se movieron  $n-i$  elementos: peor caso  $\Theta(n)$ .

## Array dinámico

Cuando se llena, reserva uno mayor y copia elementos. Una ampliación cuesta  $\Theta(n)$ , pero si la capacidad se multiplica - por ejemplo por 2 - muchas inserciones al final tienen coste amortizado  $\Theta(1)$ . Amortizado reparte las ampliaciones ocasionales entre muchas operaciones.

## Tablas hash y conjuntos



### 3.2 Función hash

Transforma una clave en una cubeta entre 0 y  $m-1$ . Debe ser rápida y repartir claves. Dos claves distintas pueden producir el mismo índice: eso es una colisión, no necesariamente un error.

### Resolver colisiones

| Método                   | Cómo                                     | Detalle                              |
|--------------------------|------------------------------------------|--------------------------------------|
| Encadenamiento           | Cada cubeta guarda una colección         | La búsqueda recorre solo esa cubeta  |
| Direccionamiento abierto | Si está ocupada, prueba otras posiciones | Necesita reglas cuidadosas al borrar |

#### SONDEO LINEAL

$m=7$  y  $h(k)=k \bmod 7$ . Insertar 10: índice 3.

Insertar 17: también 3; probamos 4.

Insertar 24: 3 ocupado, 4 ocupado; usamos 5.

Buscar 24 repite la misma secuencia 3,4,5; detenerse antes sería incorrecto.

### Factor de carga

$\alpha = n/m$  mide ocupación. Con buen hash y carga controlada, buscar, insertar y borrar tienen tiempo esperado cercano a  $\Theta(1)$ . En el peor caso, muchas colisiones pueden producir  $\Theta(n)$ . No escribas  $O(1)$  como garantía absoluta sin condiciones.

### 3.3 Conjunto como TAD

Un conjunto no tiene repetidos ni posición relevante. Operaciones: pertenece, añadir, eliminar, unión, intersección y diferencia. Puede implementarse con hash, árbol u otra estructura; el TAD no obliga a una implementación.

| Necesidad                      | Buena elección inicial                            |
|--------------------------------|---------------------------------------------------|
| Acceso frecuente por índice    | Array                                             |
| Pertenencia rápida por clave   | Tabla hash / conjunto hash                        |
| Mantener claves ordenadas      | Árbol de búsqueda equilibrado, si está disponible |
| Recorrer en orden de inserción | Secuencia u otra estructura con orden explícito   |

### Ahora tú

#### HAZLO SIN MIRAR

1. Coste de  $a[500]$  en array.
2. ¿Por qué insertar al principio cuesta  $\Theta(n)$ ?
3.  $h(k) = k \bmod 5$ : índices de 12,17,9.
4. ¿Colisión significa claves iguales?
5. Con  $m=10$  y  $n=7$ , calcula  $\alpha$ .
6. ¿Qué propiedad distingue un conjunto?

#### CORRECCIÓN EXPLICADA

1.  $\Theta(1)$ .
2. Desplaza los  $n$  elementos existentes.
3. 2,2,4; 12 y 17 colisionan.
4. No; claves distintas pueden compartir índice.
5. 0,7.
6. No contiene repetidos y su orden no forma parte del TAD.

## TEMA 4

# Estructuras de datos secuenciales

## LA PREGUNTA QUE APRENDERÁS A CONTESTAR

¿Qué estructura conserva un orden y permite trabajar en uno o ambos extremos?

## AL TERMINAR PODRÁS

Distinguir lista, pila, cola y deque; trazar operaciones y elegir implementación.

### 4.1 Secuencia

Una secuencia mantiene elementos en posiciones lógicas: primero, siguiente, anterior y último. Un array y una lista pueden representar la misma secuencia, pero sus costes cambian.

#### Array: casillas contiguas e índice directo



#### Lista enlazada: cada nodo guarda dato y enlace



| Operación                   | Array                          | Lista enlazada   |
|-----------------------------|--------------------------------|------------------|
| Acceso por posición         | $\Theta(1)$                    | $\Theta(n)$      |
| Buscar valor                | $\Theta(n)$                    | $\Theta(n)$      |
| Insertar al inicio          | $\Theta(n)$                    | $\Theta(1)$      |
| Insertar tras nodo conocido | $\Theta(n)$ por desplazamiento | $\Theta(1)$      |
| Memoria extra               | Poca por elemento              | Enlaces por nodo |

## 4.2 Lista simplemente y doblemente enlazada

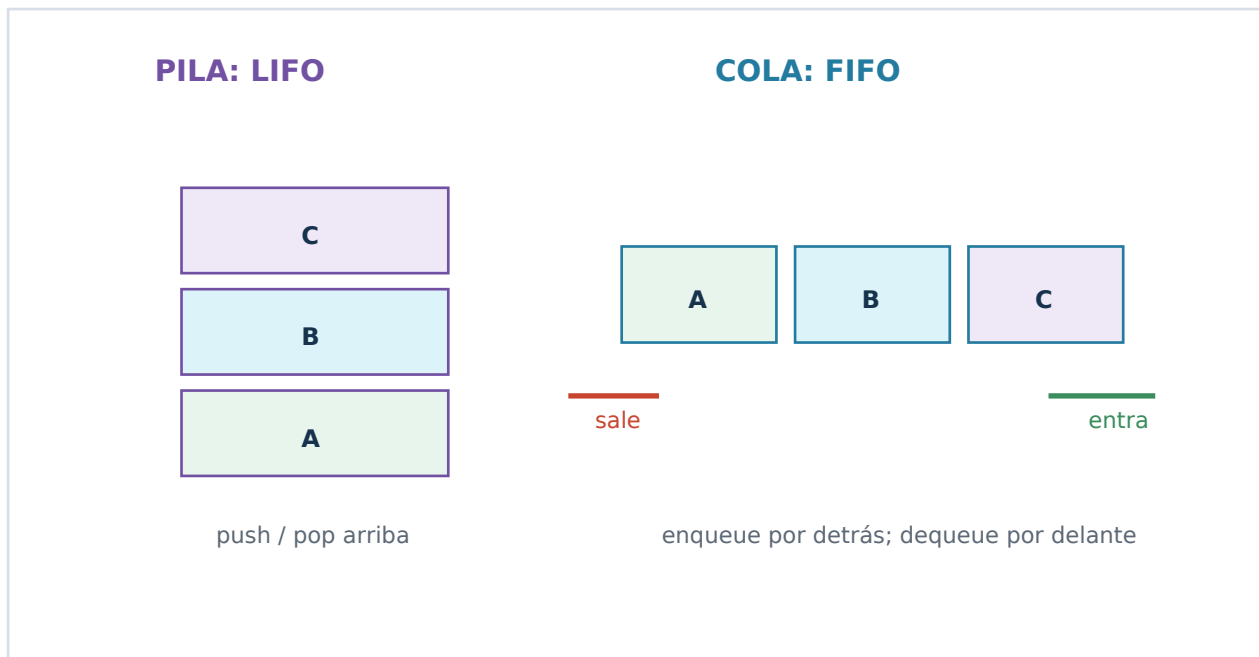
En la simple, cada nodo apunta al siguiente. En la doble, también al anterior: ocupa más memoria, pero facilita retroceder y borrar un nodo conocido. Hay que actualizar enlaces en el orden correcto para no perder la cadena.

```
insertar_inicio(x):  
    nuevo = Nodo(x)  
    nuevo.siguiente = primero  
    primero = nuevo
```

### DIBUJA ANTES DE CAMBIAR

Primero crea el nuevo enlace hacia la lista existente; después mueve primero. Si sustituyes primero antes de guardar el enlace antiguo, puedes perder el resto.

# Pilas, colas y dequeues



## 4.3 Pila

LIFO: el último en entrar es el primero en salir. Operaciones apilar, desapilar, cima y vacía. Aplicaciones: pila de llamadas, deshacer, evaluación de expresiones y recorrido en profundidad.

## 4.4 Cola

FIFO: el primero en entrar es el primero en salir. Operaciones encolar al final, desencolar al frente, primero y vacía. Aplicaciones: tareas pendientes, impresora y recorrido en amplitud.

## 4.5 Deque

Permite insertar y eliminar por ambos extremos. Una pila usa un extremo; una cola usa extremos distintos. Un buffer circular implementa colas y dequeues en array sin desplazar todos los elementos.

### COMPROBAR PARÉNTESIS

Recorremos la expresión. Si aparece (, [, {, lo apilamos.

Si aparece un cierre, la pila no puede estar vacía y la cima debe ser su apertura correspondiente.

Al final, la pila debe quedar vacía. Para ([{ }]) funciona; para ([ ]) falla al comparar ) con [.

| Estructura | Insertar       | Eliminar       | Orden de salida |
|------------|----------------|----------------|-----------------|
| Pila       | cima           | cima           | LIFO            |
| Cola       | final          | frente         | FIFO            |
| Deque      | ambos extremos | ambos extremos | según operación |

## Ahora tú

### HAZLO SIN MIRAR

1. Tras push A,B,C, ¿qué pop devuelve?
2. Encolar A,B,C y desencolar dos veces.
3. ¿Qué estructura usa comprobar paréntesis?
4. ¿Por qué una cola array ingenua no debe desplazar en cada salida?
5. Insertar al principio de lista enlazada con puntero primero: coste.

### CORRECCIÓN EXPLICADA

1. C.
2. A y después B.
3. Pila.
4. Desplazar daría  $\Theta(n)$ ; se usan índices frente/final, normalmente circulares.
5.  $\Theta(1)$ .

## TEMA 5

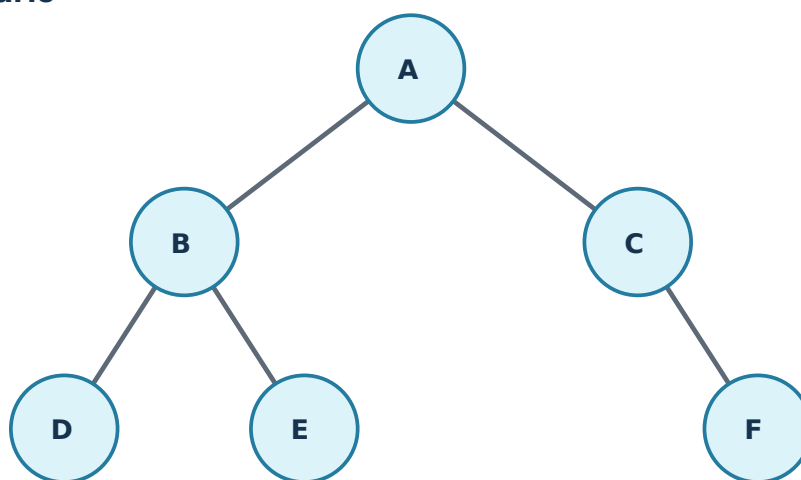
# Estructuras de datos jerárquicas

**LA PREGUNTA QUE APRENDERÁS A CONTESTAR**

¿Cómo representamos niveles, prioridades y conexiones que ya no forman una simple línea?

**AL TERMINAR PODRÁS**

Recorrer árboles, operar con BST y montículos, y representar y explorar grafos.

**Árbol binario**

Raíz A · hijos de A: B y C · hojas: D, E y F

## 5.1 Vocabulario de árboles

| Término      | Significado                             |
|--------------|-----------------------------------------|
| Raíz         | Nodo sin padre                          |
| Hijo / padre | Relación entre niveles consecutivos     |
| Hoja         | Nodo sin hijos                          |
| Subárbol     | Nodo con todos sus descendientes        |
| Profundidad  | Aristas desde la raíz hasta el nodo     |
| Altura       | Máximo número de aristas hasta una hoja |

## 5.2 Árbol general y binario

Un árbol general permite cualquier número de hijos. Uno binario permite como máximo hijo izquierdo y derecho. Binario no significa que todos tengan dos hijos, ni que esté ordenado.

## Recorridos

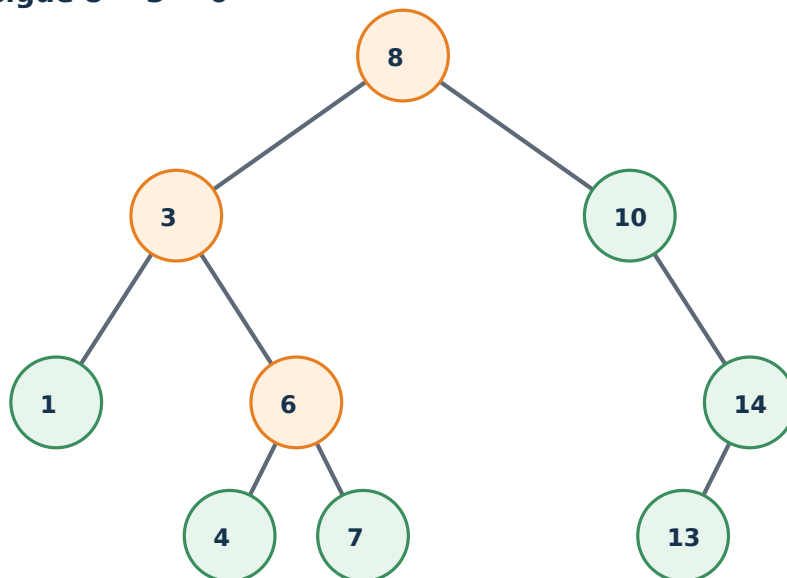
| Recorrido   | Orden conceptual         | Resultado del dibujo |
|-------------|--------------------------|----------------------|
| Preorden    | raíz, izquierda, derecha | A B D E C F          |
| Inorden     | izquierda, raíz, derecha | D B E A C F          |
| Postorden   | izquierda, derecha, raíz | D E B F C A          |
| Por niveles | nivel a nivel con cola   | A B C D E F          |

### CÓMO NO PERDERTE

Escribe la regla del recorrido arriba. Señala cada nodo exactamente cuando corresponde: antes de hijos en preorden, entre hijos en inorden y después en postorden.

## Árbol binario de búsqueda

Buscar 6 sigue 8 → 3 → 6



En un BST, las claves del subárbol izquierdo son menores y las del derecho mayores, según el criterio elegido. La forma depende del orden de inserción.

## Buscar e insertar

```

buscar(nodo, x):
  si nodo = vacío: devolver FALSO
  si x = nodo.clave: devolver VERDADERO
  si x < nodo.clave: devolver buscar(nodo.izq,x)
  devolver buscar(nodo.der,x)
  
```

El coste es  $\Theta(h)$ , donde  $h$  es la altura. Equilibrado:  $h \approx \log n$ . Degenerado como una lista:  $h \approx n$ . Por eso no debe afirmarse que todo BST busca siempre en  $O(\log n)$ .

## Borrar: tres casos

| Nodo      | Acción                                                                             |
|-----------|------------------------------------------------------------------------------------|
| Hoja      | Eliminar enlace del padre                                                          |
| Un hijo   | Conectar padre directamente con ese hijo                                           |
| Dos hijos | Sustituir por sucesor inorden - mínimo del subárbol derecho - y borrar ese sucesor |

### BORRAR 3 DEL ÁRBOL

3 tiene dos hijos. Su sucesor inorden es 4, el menor del subárbol derecho de 3.

Copiamos 4 en la posición de 3 y eliminamos el nodo 4 original, que es hoja.

La propiedad BST se conserva.

## Montículos y colas de prioridad

### Montículo máximo: cada padre $\geq$ sus hijos



Un montículo máximo es un árbol binario completo donde cada padre es mayor o igual que sus hijos. Se guarda eficientemente en array.

| Relación en array, índice i | Fórmula con índices desde 0 |
|-----------------------------|-----------------------------|
| Padre                       | $(i-1)//2$                  |
| Hijo izquierdo              | $2i+1$                      |
| Hijo derecho                | $2i+2$                      |

### INSERTAR 45

Array [50,30,40,10,20,35]. Añadimos 45 al final, índice 6.

Padre índice 2 vale 40; intercambiamos. Nuevo array [50,30,45,10,20,35,40].

El nuevo padre vale 50 y es mayor; terminamos. Coste  $\Theta(\log n)$ .

**EXTRAER MÁXIMO**

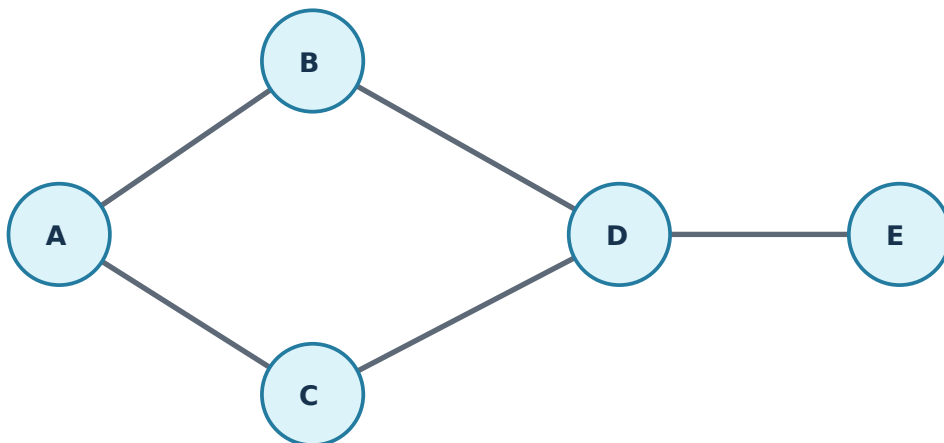
Guardamos 50, llevamos el último 40 a la raíz y eliminamos la última casilla.

Comparamos 40 con hijos 30 y 45; intercambiamos con el mayor, 45.

Queda [45,30,40,10,20,35]. Devolvemos 50. Coste  $\Theta(\log n)$ .

**Montículo no es BST**

El montículo solo ordena padre respecto a hijos; encontrar una clave cualquiera puede costar  $\Theta(n)$ . Es excelente para consultar y extraer la prioridad máxima o mínima, no para buscar cualquier valor.

**Introducción a grafos****Grafo: vértices y aristas**

BFS desde A: A, B, C, D, E · una DFS alfabética: A, B, D, C, E

**5.3 Conceptos**

| Concepto  | Significado                                 |
|-----------|---------------------------------------------|
| Vértice   | Objeto o estado                             |
| Arista    | Conexión entre vértices                     |
| Dirigido  | La conexión tiene sentido                   |
| Ponderado | La arista tiene coste o distancia           |
| Camino    | Secuencia de vértices unidos                |
| Conectado | Existe camino entre los vértices requeridos |

**Representaciones**

| Representación       | Espacio       | Consultar arista | Recorrer vecinos       |
|----------------------|---------------|------------------|------------------------|
| Matriz de adyacencia | $\Theta(V^2)$ | $\Theta(1)$      | $\Theta(V)$            |
| Lista de adyacencia  | $\Theta(V+E)$ | depende de grado | $\Theta(\text{grado})$ |

## BFS y DFS

BFS usa cola y visita por capas; en un grafo no ponderado obtiene caminos con mínimo número de aristas. DFS usa recursión o pila y profundiza antes de retroceder. Ambos deben marcar visitados para no repetir ciclos.

```
BFS(origen):  
  marcar origen; encolar origen  
  mientras cola no vacía:  
    v = desencolar  
    para cada vecino w de v:  
      si w no marcado: marcar w; encolar w
```

Con listas de adyacencia, BFS y DFS cuestan  $\Theta(V+E)$ : cada vértice se marca una vez y cada arista se examina un número constante de veces.

## Ahora tú

### HAZLO SIN MIRAR

1. Preorden del árbol A-B/C, B-D/E, C-F.
2. Camino de búsqueda de 7 en el BST dibujado.
3. Coste de buscar en BST degenerado.
4. En heap máximo, ¿dónde está el máximo?
5. BFS usa ¿pila o cola?
6. Espacio de matriz para V vértices.

### CORRECCIÓN EXPLICADA

1. A,B,D,E,C,F.
2.  $8 \rightarrow 3 \rightarrow 6 \rightarrow 7$ .
3.  $\Theta(n)$ .
4. En la raíz, índice 0.
5. Cola.
6.  $\Theta(V^2)$ .

# Banco de 45 ejercicios progresivos

Escribe trazas y dibujos. Una respuesta sin justificación no entrena el examen de desarrollo.

| N.º | Tema      | Ejercicio                                          |
|-----|-----------|----------------------------------------------------|
| 1   | Puente    | Traza buscar([6,2,9,4],9).                         |
| 2   | Puente    | Escribe la invariante de búsqueda lineal.          |
| 3   | TAD       | Diferencia interfaz e implementación con una pila. |
| 4   | Recursión | Traza factorial(5).                                |
| 5   | Recursión | Suma [4,1,7] con sumar(a,n).                       |
| 6   | Recursión | Detecta el fallo: $f(n)=f(n+1)$ con base $n=0$ .   |
| 7   | Recursión | Binaria busca 31 en [2,7,12,20,31,40,55].          |
| 8   | Recursión | Tiempo y pila de longitud de lista.                |
| 9   | Recursión | Demuestra terminación de cuenta atrás.             |
| 10  | Coste     | Simplifica $8n^2+3n+50$ .                          |
| 11  | Coste     | Dos bucles consecutivos n y 4n.                    |
| 12  | Coste     | Bucles anidados n y n.                             |
| 13  | Coste     | Bucle $i*=3$ hasta n.                              |
| 14  | Coste     | Triangular $1+\dots+n$ .                           |
| 15  | Coste     | $T(n)=T(n/2)+1$ .                                  |
| 16  | Coste     | Distingue mejor y peor caso lineal.                |
| 17  | Coste     | Espacio de matriz $n \times n$ .                   |
| 18  | Array     | Inserta 8 en índice 1 de [3,5,9].                  |
| 19  | Array     | Borra índice 2 de [4,7,6,1].                       |
| 20  | Array     | Compara acceso por índice y búsqueda por valor.    |
| 21  | Hash      | $h(k)=k \bmod 8$ para 10,18,25.                    |
| 22  | Hash      | Sondeo lineal de 10,18,26 con $m=8$ .              |
| 23  | Hash      | $\alpha$ con 12 claves y 16 cubetas.               |

## Banco de ejercicios: 24-45

| N.º | Tema     | Ejercicio                                           |
|-----|----------|-----------------------------------------------------|
| 24  | Hash     | Explica peor caso $\Theta(n)$ .                     |
| 25  | Conjunto | Unión e intersección de $\{1,2,4\}$ y $\{2,3,4\}$ . |
| 26  | Lista    | Inserta X al inicio dibujando enlaces.              |
| 27  | Lista    | Por qué buscar índice 50 cuesta $\Theta(n)$ .       |
| 28  | Pila     | Push 2,5,7; pop; top.                               |
| 29  | Cola     | Enqueue A,B,C; dequeue; enqueue D.                  |
| 30  | Deque    | Operaciones para producir C,A,B.                    |
| 31  | Pila     | Comprueba $\{[(())]\}$ y $\{[(())]\}$ .             |
| 32  | Cola     | Diseña cola circular de capacidad 5.                |
| 33  | Árbol    | Identifica raíz, hojas y altura del dibujo.         |
| 34  | Árbol    | Inorden y postorden.                                |
| 35  | BST      | Inserta 5 en el BST dibujado.                       |
| 36  | BST      | Borra hoja 1.                                       |
| 37  | BST      | Borra 10, que tiene un hijo.                        |
| 38  | BST      | Borra 3, que tiene dos hijos.                       |
| 39  | Heap     | Inserta 45 en [50,30,40,10,20,35].                  |
| 40  | Heap     | Extrae máximo del resultado.                        |
| 41  | Heap     | Índices de hijos de $i=4$ .                         |
| 42  | Grafo    | Lista de adyacencia del dibujo.                     |
| 43  | Grafo    | BFS desde A.                                        |
| 44  | Grafo    | Una DFS alfabética desde A.                         |
| 45  | Grafo    | Coste BFS con listas.                               |

# Soluciones razonadas: 1-23

| N.º | Corrección                                                                                  |
|-----|---------------------------------------------------------------------------------------------|
| 1   | $i=0:6$ no; $i=1:2$ no; $i=2:9$ sí; devuelve 2.                                             |
| 2   | Antes de $i$ , el objetivo no aparece en $0..i-1$ .                                         |
| 3   | TAD: push/pop/top; implementación posible: array o lista.                                   |
| 4   | Llamadas 5,4,3,2,1,0; retornos 1,1,2,6,24,120.                                              |
| 5   | $7+1+4+0=12$ .                                                                              |
| 6   | $n$ crece y no alcanza 0; no hay progreso.                                                  |
| 7   | Centro índice3=20; queda4..6; centro5=40; queda4..4; índice4=31.                            |
| 8   | Tiempo $\Theta(n)$ , pila $\Theta(n)$ .                                                     |
| 9   | Para $n \geq 0$ , cada llamada reduce $n$ en 1 y los naturales no decrecen indefinidamente. |
| 10  | $\Theta(n^2)$ .                                                                             |
| 11  | $n+4n=5n=\Theta(n)$ .                                                                       |
| 12  | $n \cdot n = \Theta(n^2)$ .                                                                 |
| 13  | $\Theta(\log_3 n) = \Theta(\log n)$ .                                                       |
| 14  | $n(n+1)/2 = \Theta(n^2)$ .                                                                  |
| 15  | $\Theta(\log n)$ .                                                                          |
| 16  | Mejor $\Theta(1)$ si está primero; peor $\Theta(n)$ si está al final o no está.             |
| 17  | $\Theta(n^2)$ .                                                                             |
| 18  | Mover 9 y 5 a la derecha; [3,8,5,9].                                                        |
| 19  | Eliminar 6 y mover 1; [4,7,1].                                                              |
| 20  | $a[i] \Theta(1)$ ; buscar valor sin ordenar $\Theta(n)$ .                                   |
| 21  | 2,2,1; 10 y 18 colisionan.                                                                  |
| 22  | 10→2; 18 prueba2 y usa3; 26 prueba2,3 y usa4.                                               |
| 23  | $\alpha=12/16=0,75$ .                                                                       |

# Soluciones razonadas: 24-45

| N.º | Corrección                                                                         |
|-----|------------------------------------------------------------------------------------|
| 24  | Si todas las claves caen en la misma cubeta, se recorre una cadena de n elementos. |
| 25  | Unión={1,2,3,4}; intersección={2,4}.                                               |
| 26  | nuevo.sig=primero; primero=nuevo. Coste $\Theta(1)$ .                              |
| 27  | No existe cálculo directo; hay que seguir 50 enlaces desde el inicio.              |
| 28  | Pila [2,5,7]; pop devuelve7; top queda5.                                           |
| 29  | Sale A; queda B,C y entra D: B,C,D.                                                |
| 30  | Una posibilidad: añadir A atrás, B atrás, C delante; queda C,A,B.                  |
| 31  | {[()] } termina vacía y es válida; {[()] } falla al cerrar ] con cima (.           |
| 32  | Usa array, frente, tamaño y final=(frente+tamaño) mod 5; no desplaza.              |
| 33  | Raíz A; hojas D,E,F; altura 2 aristas.                                             |
| 34  | Inorden D,B,E,A,C,F; postorden D,E,B,F,C,A.                                        |
| 35  | 5: 8→3→6; como5<6 y >3, hijo izquierdo de6.                                        |
| 36  | Se elimina el enlace izquierdo de3.                                                |
| 37  | 10 se sustituye por su único hijo14.                                               |
| 38  | Sucesor de3 es4; sustituir y borrar hoja4.                                         |
| 39  | [50,30,45,10,20,35,40].                                                            |
| 40  | Extrae50; queda [45,30,40,10,20,35].                                               |
| 41  | $2 \cdot 4 + 1 = 9$ y $2 \cdot 4 + 2 = 10$ .                                       |
| 42  | A:{B,C}; B:{A,D}; C:{A,D}; D:{B,C,E}; E:{D}.                                       |
| 43  | A,B,C,D,E.                                                                         |
| 44  | Con vecinos alfabéticos: A,B,D,C,E.                                                |
| 45  | $\Theta(V+E)$ .                                                                    |

## DIAGNÓSTICO

Si fallan 1-9, refuerza trazas y recursión. Si fallan 10-17, vuelve a reglas de coste. Si fallan 18-32, dibuja memoria y extremos. Si fallan 33-45, repite recorridos sin mirar.

# Ocho modelos de respuesta de desarrollo

## 1. Recursión

**Enunciado:** Defina factorial, trace factorial(4) y justifique terminación.

### RESPUESTA MODELO

Base  $n=0$ ; paso  $n \cdot \text{fact}(n-1)$ ; llamadas 4..0; retornos 1,1,2,6,24. Termina porque  $n$  natural disminuye estrictamente hasta 0.

## 2. Coste

**Enunciado:** Analice un bucle triangular.

### RESPUESTA MODELO

Total  $\sum_i$  de 0 a  $n-1 = n(n-1)/2$ ; por tanto  $\Theta(n^2)$ . Espacio auxiliar  $\Theta(1)$  si solo usa contadores.

## 3. Array

**Enunciado:** Inserte en el medio y justifique coste.

### RESPUESTA MODELO

Calcula hueco, desplaza de derecha a izquierda y escribe. Mueve hasta  $n$  elementos:  $\Theta(n)$ ; acceso al índice sigue  $\Theta(1)$ .

## 4. Hash

**Enunciado:** Inserte claves con encadenamiento y discuta coste.

### RESPUESTA MODELO

Calcula  $h$  para cada clave, dibuja cubetas y colisiones. Esperado  $\Theta(1)$  con buen reparto/carga; peor  $\Theta(n)$  si se concentran.

# Modelos de desarrollo: 5-8

## 5. Lista y pila

**Enunciado:** Implemente pila con lista.

### RESPUESTA MODELO

push inserta al inicio; pop guarda dato, mueve primero a siguiente y devuelve. Ambos  $\Theta(1)$ ; controlar pila vacía.

## 6. BST

**Enunciado:** Busque y borre un nodo con dos hijos.

### RESPUESTA MODELO

Sigue comparaciones  $\Theta(h)$ . Para borrar, usa sucesor inorden, copia clave y elimina sucesor; demuestra que se conserva el orden.

## 7. Montículo

**Enunciado:** Inserte y extraiga prioridad máxima.

### RESPUESTA MODELO

Insertar al final y subir; extraer raíz, mover último y bajar con mayor hijo. Ambos  $\Theta(\log n)$ ; máximo  $\Theta(1)$ .

## 8. Grafo

**Enunciado:** Recorra con BFS y analice.

### RESPUESTA MODELO

Cola, visitados y vecinos. Cada vértice se marca una vez y cada arista se examina constantemente:  $\Theta(V+E)$  con listas.

# Simulacro de examen de desarrollo

Tiempo: 120 minutos, sin material. La guía pública no fija el número de preguntas; usamos cinco ejercicios para practicar una combinación completa del temario.

| Ejercicio | Tiempo orientativo | Puntos de entrenamiento |
|-----------|--------------------|-------------------------|
| 1         | 20 min             | 2                       |
| 2         | 25 min             | 2                       |
| 3         | 20 min             | 2                       |
| 4         | 25 min             | 2                       |
| 5         | 20 min             | 2                       |
| Revisión  | 10 min             | -                       |

## 1. Recursión

Escriba `suma_rec(a,n)`, trace la llamada con `[2,4,3]` y `n=3`, justifique terminación y calcule tiempo y espacio.

## 2. Coste

Analice tiempo y espacio del algoritmo: para  $i=1..n$ ,  $j=1$ ; mientras  $j \leq n$ : operación;  $j=2j$ . Explique cada regla.

## 3. Array y hash

a) Inserte 9 en índice 2 de `[3,7,12,20]` y calcule coste. b) En tabla de 7 cubetas con  $h(k)=k \bmod 7$  y encadenamiento, inserte 8,15,22,10 y muestre la tabla. Discuta costes.

## 4. Secuenciales

a) Muestre el estado de una pila tras push A,B,C, pop, push D. b) Diseñe el algoritmo de paréntesis equilibrados y aplíquelo a `{[()]}` y `{[(())]}`. c) Justifique complejidad.

## 5. Jerárquicas

Con el árbol del manual: a) preorden, inorden, postorden y niveles; b) busque 13 en el BST; c) inserte 45 en el montículo; d) BFS del grafo desde A y costes.

# Corrección del simulacro: 1-2

## 1. Recursión

```
suma_rec(a,n):
    si n=0: devolver 0
    devolver a[n-1] + suma_rec(a,n-1)

suma(a,3)=3+suma(a,2)=3+4+suma(a,1)=3+4+2+suma(a,0)=9
```

Termina porque  $n$  natural disminuye en 1 hasta 0. Hace  $\Theta(n)$  llamadas/trabajo y mantiene  $\Theta(n)$  marcos de pila.

## 2. Coste

El bucle exterior se repite  $n$  veces. En cada repetición,  $j$  toma 1,2,4,... hasta superar  $n$ :  $\Theta(\log n)$  iteraciones. Se multiplican:  $\Theta(n \log n)$ . Solo usa  $i$  y  $j$ , luego espacio auxiliar  $\Theta(1)$ .

### JUSTIFICACIÓN COMPLETA

No basta escribir  $O(n \log n)$ . Deben aparecer las  $n$  repeticiones exteriores, la duplicación de  $j$  y la multiplicación de costes.

# Corrección del simulacro: 3-4

## 3. Array y hash

Array: mover 20 y 12 una posición a la derecha y escribir 9: [3,7,9,12,20]. Inserción en medio  $\Theta(n)$ , acceso por índice  $\Theta(1)$ .

```
m=7, h(k)=k mod 7
8 -> 1
15 -> 1    cubeta 1: 8 -> 15 -> 22
22 -> 1
10 -> 3    cubeta 3: 10
```

Con buen reparto y carga controlada, operaciones esperadas cercanas a  $\Theta(1)$ ; peor  $\Theta(n)$  si todas las claves colisionan.

## 4. Secuenciales

Pila:  $[] \rightarrow [A] \rightarrow [A,B] \rightarrow [A,B,C] \rightarrow \text{pop } C = [A,B] \rightarrow \text{push } D = [A,B,D]$ ; la cima es  $D$ .

Paréntesis: apilar aperturas; al cerrar, exigir cima correspondiente y desapilar; al final exigir pila vacía.  $\{[()]\}$  es válida.  $\{[(())]\}$  falla cuando  $]$  encuentra  $($  en la cima. Cada símbolo entra y sale como máximo una vez:  $\Theta(n)$  tiempo y  $\Theta(n)$  espacio en peor caso.

# Corrección del simulacro: 5

## Recorridos del árbol

| Recorrido | Resultado   |
|-----------|-------------|
| Preorden  | A B D E C F |
| Inorden   | D B E A C F |
| Postorden | D E B F C A |
| Niveles   | A B C D E F |

## BST

Buscar 13:  $8 \rightarrow 10 \rightarrow 14 \rightarrow 13$ . Cuatro nodos visitados. Coste  $\Theta(h)$ .

## Montículo

Añadir 45 al final de [50,30,40,10,20,35]; comparar con padre 40 e intercambiar; detener ante 50. Resultado [50,30,45,10,20,35,40],  $\Theta(\log n)$ .

## Grafo

BFS alfabética desde A: A,B,C,D,E. Usa cola y conjunto de visitados. Con lista de adyacencia,  $\Theta(V+E)$  tiempo y  $\Theta(V)$  espacio auxiliar.

### AUTOCORRECCIÓN SEGÚN LA GUÍA

Comprueba cuatro columnas mentales: ¿es correcto?, ¿está completo?, ¿todo es relevante?, ¿cada afirmación importante está justificada? Deja margen y escribe con bolígrafo en el examen real.

# Hoja de memoria

| Estructura  | Acceso/búsqueda                         | Inserción clave               | Idea                       |
|-------------|-----------------------------------------|-------------------------------|----------------------------|
| Array       | índice $\Theta(1)$ ; valor $\Theta(n)$  | medio $\Theta(n)$             | contiguo                   |
| Hash        | esperado $\Theta(1)$ ; peor $\Theta(n)$ | igual                         | clave $\rightarrow$ cubeta |
| Lista       | posición/valor $\Theta(n)$              | inicio $\Theta(1)$            | nodos y enlaces            |
| Pila        | cima $\Theta(1)$                        | push $\Theta(1)$              | LIFO                       |
| Cola        | frente $\Theta(1)$                      | enqueue $\Theta(1)$           | FIFO                       |
| BST         | $\Theta(h)$                             | $\Theta(h)$                   | izq < raíz < der           |
| Heap        | máximo $\Theta(1)$                      | $\Theta(\log n)$              | padre domina hijos         |
| Grafo lista | arista depende del grado                | añadir usualmente $\Theta(1)$ | $\Theta(V+E)$ espacio      |

## Patrones de coste

| Patrón                            | Orden              |
|-----------------------------------|--------------------|
| Una operación                     | $\Theta(1)$        |
| Recorrer n                        | $\Theta(n)$        |
| Dos bucles $n \times n$           | $\Theta(n^2)$      |
| Dividir por 2                     | $\Theta(\log n)$   |
| Dividir y combinar lineal         | $\Theta(n \log n)$ |
| Árbol/grafos completos con listas | $\Theta(V+E)$      |

### REGLA DE ORO

El coste depende de la operación, la implementación y las condiciones. Escribe siempre qué representa n, qué caso analizas y por qué se repite ese número de veces.

# Método para el examen

| Minutos | Acción                                                  |
|---------|---------------------------------------------------------|
| 0-5     | Leer todo, marcar datos y elegir orden.                 |
| 5-100   | Resolver por bloques; reservar tiempo según puntuación. |
| 100-112 | Completar justificaciones, costes y casos frontera.     |
| 112-120 | Revisar trazas, índices, enlaces y margen de escaneo.   |

## Plantilla de respuesta algorítmica

Objetivo y datos:  
Estructura elegida y por qué:  
Algoritmo/pseudocódigo:  
Traza con el ejemplo:  
Corrección y terminación:  
Tiempo y espacio, indicando n:  
Casos frontera:

## Errores que debes evitar

- Decir que recursivo significa automáticamente lento.
- Confundir  $O(n)$  con n segundos.
- Afirmar  $O(1)$  para hash sin mencionar promedio, carga y colisiones.
- Dar costes de lista como si tuviera acceso directo.
- Confundir pila LIFO con cola FIFO.
- Llamar BST a cualquier árbol binario.
- Confundir heap con árbol totalmente ordenado.
- Recorrer un grafo sin conjunto de visitados.

### PRÁCTICA OBLIGATORIA

Contacta con el tutor o Centro Asociado de Lugo con antelación para calendario y sesiones. La práctica debe ser propia, entregarse en plazo y superar los juegos de prueba. No se usará IA generativa para elaborarla salvo autorización expresa del equipo docente.

# Plan de estudio en 55 sesiones

| Sesiones | Bloque     | Meta                                        |
|----------|------------|---------------------------------------------|
| 1-4      | Puente     | Funciones, referencias, POO y trazas        |
| 5-12     | Tema 1     | Recursión, pila, binaria y corrección       |
| 13-22    | Tema 2     | RAM, Landau, bucles, recurrencias y espacio |
| 23-30    | Tema 3     | Arrays, hash, colisiones y conjuntos        |
| 31-38    | Tema 4     | Listas, pilas, colas, deque y aplicaciones  |
| 39-48    | Tema 5     | Árboles, BST, montículos y grafos           |
| 49-51    | Banco      | 45 ejercicios y registro de errores         |
| 52-53    | Desarrollo | Ocho modelos sin mirar                      |
| 54       | Simulacro  | 120 minutos                                 |
| 55       | Corrección | Reescribir respuestas débiles               |

## Ritual de 70 minutos

| Minutos                                                                                                                                          | Acción                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| 0-10                                                                                                                                             | Recuperar definición, dibujo y coste sin mirar. |
| 10-25                                                                                                                                            | Aprender una idea con un ejemplo pequeño.       |
| 25-50                                                                                                                                            | Trazar o programar ejercicios de entrenamiento. |
| 50-60                                                                                                                                            | Corregir en otro color y clasificar el error.   |
| 60-70                                                                                                                                            | Responder un mini desarrollo completo.          |
| <b>AVANZA CUANDO</b>                                                                                                                             |                                                 |
| Puedes dibujar la estructura, ejecutar las operaciones, justificar por qué termina y calcular tiempo y espacio sin repetir una frase memorizada. |                                                 |

# Glosario esencial

| Término        | Definición sencilla                                              |
|----------------|------------------------------------------------------------------|
| Algoritmo      | Secuencia precisa y finita de pasos.                             |
| Array          | Colección contigua con acceso por índice.                        |
| BST            | Árbol binario que ordena claves por subárboles.                  |
| Cola           | TAD FIFO: entra atrás y sale delante.                            |
| Colisión       | Dos claves hash producen la misma cubeta.                        |
| Complejidad    | Crecimiento de tiempo o memoria con la entrada.                  |
| Deque          | Cola doble con operaciones en ambos extremos.                    |
| Grafo          | Vértices conectados por aristas.                                 |
| Hash           | Función que transforma clave en índice.                          |
| Heap           | Árbol completo donde cada padre domina a sus hijos.              |
| Invariante     | Propiedad cierta durante el algoritmo.                           |
| Lista enlazada | Secuencia de nodos conectados por referencias.                   |
| Pila           | TAD LIFO: entra y sale por la cima.                              |
| Recursión      | Solución que usa una instancia menor del mismo problema.         |
| TAD            | Comportamiento definido independientemente de la implementación. |
| $\Theta$       | Cota ajustada de crecimiento.                                    |

## Fuente y alcance

- Guía de estudio pública UNED 2026/27 de Estructuras de Datos y Algoritmos, código 71032015, para requisitos, temario y evaluación.
- Explicaciones, pseudocódigo, figuras, ejercicios y simulacro de elaboración didáctica original para estudio.
- No se encontraron exámenes históricos identificados de esta nueva asignatura en las carpetas revisadas.

El equipo docente proporcionará apuntes y las instrucciones concretas de la práctica en el curso virtual. Ese material prevalece y deberá incorporarse cuando se publique.

### PRIMERA META

Estudia el puente y el Tema 1. Debes poder trazar factorial y suma recursiva, señalar caso base y progreso, y explicar por qué la pila crece.

**Una buena estructura no hace magia: convierte la operación importante en un camino corto y justificable.**